

Exercise 10

Last update April 9, 2026

This exercise sheet must be handed in via LearnIt.

You are expected to solve assignments in your group.

Your name (or ITU logins) must be part of the filename, e.g., FP-GC-<name1>_<name2>.fsx (or .java).

An example: FP-GC-MadsAndersen_HansNielsen.fsx.

You can only upload one file.

It is important that you annotate your own code with comments.

You get 1 point for each assignment. Suggest you prioritize assignments as ordered below, that is, do the exam assignments before the queue problem.

Exercise 10.1 Do assignment 1 in exam set from June 2018.

Exercise 10.2 Do assignment 2 in exam set from June 2018. YOU SHOULD NOT DO THE QUESTIONS INVOLVING PARALLEL PROGRAMMING.

Exercise 10.3 We will study an example Queue implementation that has a memory leak such that the garbage collector doesn't free memory that is actually not needed for the rest of the computation.

The original example is found in the Java file `QueueWithMistake.java` found in the code repository for lecture 10. You also find the file `QueueWithMistake.fsx` which is a direct translation into F#. The translation shows imperative features of F# and is very similar to the Java implementation which also means that the solution to the space leak is the same for the two implementations. As a side note there are not only one way to fix the problem.

What you need to do is

1. Chose either the Java or F# program and run it as shown below. The program runs out of memory on my computer, it might not on yours unless you increase the iterations and number of queues.

```
% javac QueueWithMistake.java
% java -verbosegc QueueWithMistake

[0.004s][info][gc] Using G1
[0.047s][info][gc] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 25M->24M(584M) 16,577ms
[0.060s][info][gc] GC(1) Pause Young (Normal) (G1 Evacuation Pause) 40M->42M(584M) 10,351ms
[0.072s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 58M->61M(584M) 9,875ms
[0.084s][info][gc] GC(3) Pause Young (Normal) (G1 Evacuation Pause) 77M->77M(584M) 9,967ms
[0.104s][info][gc] GC(4) Pause Young (Normal) (G1 Evacuation Pause) 101M->104M(1752M) 16,083ms
[0.167s][info][gc] GC(5) Pause Young (Normal) (G1 Evacuation Pause) 176M->176M(1752M) 46,537ms
[0.218s][info][gc] GC(6) Pause Young (Normal) (G1 Evacuation Pause) 240M->245M(1752M) 41,485ms
[0.271s][info][gc] GC(7) Pause Young (Normal) (G1 Evacuation Pause) 309M->314M(1752M) 43,310ms
[0.343s][info][gc] GC(8) Pause Young (Normal) (G1 Evacuation Pause) 402M->405M(4744M) 58,138ms
SentinelLockQueue Qno. 0 0,34 19999999
...

[23.679s][info][gc] GC(50) Pause Young (Normal) (G1 Evacuation Pause) 9211M->9211M(9216M) 0,295ms
[23.687s][info][gc] GC(51) Pause Full (G1 Compaction Pause) 9211M->1M(160M) 7,685ms
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
at SentinelLockQueue.put(QueueWithMistake.java:81)
at QueueWithMistake.run(QueueWithMistake.java:41)
at QueueWithMistake.main(QueueWithMistake.java:29)
%
```

The F# version completes on my computer but clearly takes time and memory to complete.

```
% dotnet fsx

Microsoft (R) F# Interactive version 12.8.200.0 for F# 8.0
Copyright (c) Microsoft Corporation. All Rights Reserved.

For help type #help;;

> #time;;

--> Timing now on

> #load "QueueWithMistake.fsx";;
[Loading /Users/nh/Dropbox/Documents/Work/ITU/Course/KSFUPRO1KU-F2024/Handin/Handin1/QueueWithMistake.fsx]
Real: 00:00:00.000, CPU: 00:00:00.000, GC gen0: 0, gen1: 0, gen2: 0
module FSI_0002.QueueWithMistake
type Node =
  new: item: int * next: Node option -> Node
  member Item: int with get, set
  member Next: Node option with get, set
type SentinelLockQueue =
  new: unit -> SentinelLockQueue
  member get: unit -> int
  member put: item: int -> unit
val time: f: (unit -> 'a) -> 'a * System.TimeSpan
val run: unit -> unit
```

```
> open QueueWithMistake;;
Real: 00:00:00.000, CPU: 00:00:00.000, GC gen0: 0, gen1: 0, gen2: 0

> run();;
Qno. 0 19999999 00:00:00.3297830
Qno. 1 19999999 00:00:00.3206560
Qno. 2 19999999 00:00:00.4223970
...
Qno. 47 19999999 00:00:00.4290960
Qno. 48 19999999 00:00:01.8087370
Qno. 49 19999999 00:00:00.4750640

Real: 00:00:33.318, CPU: 00:00:01.975, GC gen0: 111, gen1: 66, gen2: 5
val it: unit = ()

>
```

It has performed 111 minor, 66 generation 1 and 5 major garbage collections without being able to free much memory – and it has taken some time. My laptop has 36Gb of memory which is used.

2. Study the code and find out what the problem is, explain it, and fix it.
3. The corrected code should run to completion without error

If you fix the problem the solution will run faster. Below the fixed Java version:

```
$ javac QueueWithMistakeSolved2.java
$ java -verbosegc QueueWithMistakeSolved2

h@MBP-Niels Handin11 % java -verbosegc QueueWithMistakeSolved2
[0.004s][info][gc] Using G1
[0.031s][info][gc] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 25M->1M(584M) 0,606ms
[0.103s][info][gc] GC(1) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,426ms
SentinelLockQueue Qno. 0 0,10 19999999
[0.167s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,470ms
SentinelLockQueue Qno. 1 0,18 19999999
[0.214s][info][gc] GC(3) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,492ms
[0.260s][info][gc] GC(4) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,497ms
SentinelLockQueue Qno. 2 0,24 19999999
...
[3.143s][info][gc] GC(66) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,263ms
SentinelLockQueue Qno. 48 3,16 19999999
[3.191s][info][gc] GC(67) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,287ms
[3.237s][info][gc] GC(68) Pause Young (Normal) (G1 Evacuation Pause) 337M->1M(584M) 0,342ms
SentinelLockQueue Qno. 49 3,22 19999999
```

Below the fixed F# version:

```
% dotnet fsi

Microsoft (R) F# Interactive version 12.8.200.0 for F# 8.0
Copyright (c) Microsoft Corporation. All Rights Reserved.

For help type #help;;

> #time;;

--> Timing now on

> #load "QueueWithMistakeSolved.fsx";;
[Loading /Users/nh/Dropbox/Documents/Work/ITU/Course/KSFUPRO1KU-F2024/Handin/Handin11/QueueWithMistakeSolved.fsx]
Real: 00:00:00.000, CPU: 00:00:00.000, GC gen0: 0, gen1: 0, gen2: 0
module FSI_0002.QueueWithMistakeSolved
type Node =
    new: item: int * next: Node option -> Node
    member Item: int with get, set
    member Next: Node option with get, set
...

val run: unit -> unit

> open QueueWithMistakeSolved;;
Real: 00:00:00.000, CPU: 00:00:00.000, GC gen0: 0, gen1: 0, gen2: 0
> run();;
Qno. 0 19999999 00:00:00.3522810
Qno. 1 19999999 00:00:00.3443960
Qno. 2 19999999 00:00:00.4700500
Qno. 3 19999999 00:00:00.3306260
...
Qno. 47 19999999 00:00:00.6283460
Qno. 48 19999999 00:00:03.5234750
Qno. 49 19999999 00:00:00.2654650
```

```
Real: 00:00:18.267, CPU: 00:00:00.541, GC gen0: 249, gen1: 224, gen2: 0  
val it: unit = ()
```

>

Notice, that there has not been the need for major collection, `gen2: 0`.

You find more explanation in the files `QueueWithMistake.java` and `QueueWithMistake.fsx`.